SPENDWISE

Backend Development with Node.js by MongoDB

Project Documentation

Table of Contents

1. Introduction..... 1
2. Project Overview..... 2
3. Key Objectives..... 2
4. Target Users..... 2
5. Pre-Requisites..... 2
6. Project Architecture..... 2
7. Technical Architecture..... 3
8. Database Design..... 3
9. Features..... 3
10. Roles and Responsibilities..... 3
11. Project Setup..... 3
12. Folder Structure..... 4
13. API Endpoints..... 4
14. API Testing..... 5
15. Project Execution..... 5
16. Screenshots or Demo..... 5
17. Conclusion..... 5

1. Introduction

- **Project Title:** SpendWise – A Simple Backend Learning Project
- **Team ID:** 69a6a0e2c3309e5746ec1bc1
- **Team Leader:** Balaji M [balajimani264@gmail.com]
- **Team Members:**
  - Sakthi P [palanisamyp55895@gmail.com]
  - Sakthivel G [sakthivelmegala820@gmail.com]
  - Sanjay K [sanjaykumat7427@gmail.com]
  - Santhosh Kumar S [santhoshskm2006@gmail.com]

2. Project Overview

SpendWise is a backend application designed to help beginner developers learn how to build secure REST APIs using Node.js, Express, MongoDB, and JWT authentication. The project focuses on implementing user authentication and basic CRUD operations in a clean and simple way so students can understand how backend systems work in real-world scenarios.

3. Key Objectives

- Provide secure user authentication using JWT.
- Enable basic CRUD operations on user data (transactions/records).
- Demonstrate MongoDB integration using Mongoose.
- Show how protected routes work using middleware.
- Maintain a simple and beginner-friendly backend architecture.

4. Target Users

- Beginner backend developers.
- Students learning Node.js and Express.
- Freshers preparing for technical interviews.
- Anyone who wants a simple project to understand JWT and MongoDB.
- Learners who want hands-on practice with REST APIs.

5. Pre-Requisites

Software Requirements:

Software	Purpose
Node.js	JavaScript runtime for backend
MongoDB	NoSQL database for data storage
npm	Package manager
Git	Version control system
Code Editor	VS Code
Thunder Client	To test and run APIs

Knowledge Requirements:

- JavaScript ES6+
- Node.js and Express.js
- MongoDB and Mongoose
- JWT authentication
- Basic REST API concepts
- Git basics.

6. Project Architecture

The system handles HTTP requests from clients like Postman or Thunder Client. The Express server routes these requests to controllers, which interact with MongoDB via Mongoose models. A JWT Middleware intercepts requests to verify tokens and protect private APIs before allowing CRUD queries to proceed to the database.

7. Technical Architecture

- **Controllers:** Handle business logic for authentication and CRUD.
- **Models:** Mongoose schemas for User and Transaction.
- **Routes:** API endpoints for auth and CRUD operations.
- **Middleware:** JWT verification and request protection.
- **Configuration:** Database connection and environment variables.

8. Database Design

Users Collection:

_id, name, email, password (hashed), createdAt.

Transactions Collection:

_id, userId (FK), title/description, amount/value, createdAt.

Relationship: One User has Many Transactions.

9. Features

- **User Authentication:** JWT-based registration and login.
- **CRUD Operations:** Create, Read, Update, Delete transactions.
- **Protected Routes:** Secure APIs using JWT middleware.
- **MongoDB Integration:** Store and fetch data using Mongoose.

10. Roles and Responsibilities

Student / Developer:

- Write backend code and create APIs.
- Test APIs using Postman/Thunder Client.
- Push code to GitHub.

System (Backend):

- Authenticate users using JWT.
- Handle CRUD operations.
- Store and fetch data from MongoDB.
- Protect private routes.

11. Project Setup

1. Choose or create a directory for the backend project.
2. Open terminal and navigate to that directory.
3. Initialize Node.js project using npm init.
4. Install required packages: express, mongoose, jsonwebtoken, bcrypt, cors, dotenv.
5. Create folders: models, routes, controllers, middleware, config.
6. Create a .env file and define PORT and DB URL.

12. Folder Structure

```
SPENDWISE/
├── config/
│   └── db.js
├── controllers/
│   ├── authController.js
│   └── transactionController.js
├── middleware/
│   └── authMiddleware.js
├── models/
│   ├── Transaction.js
│   └── User.js
├── routes/
│   ├── authRoutes.js
│   └── transactionRoutes.js
├── .env
├── package.json
└── server.js
```

13. API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register new user
POST	/api/auth/login	Login user
POST	/api/transactions	Create transaction
GET	/api/transactions	Read transactions
PUT	/api/transactions/:id	Update transaction
DELETE	/api/transactions/:id	Delete transaction

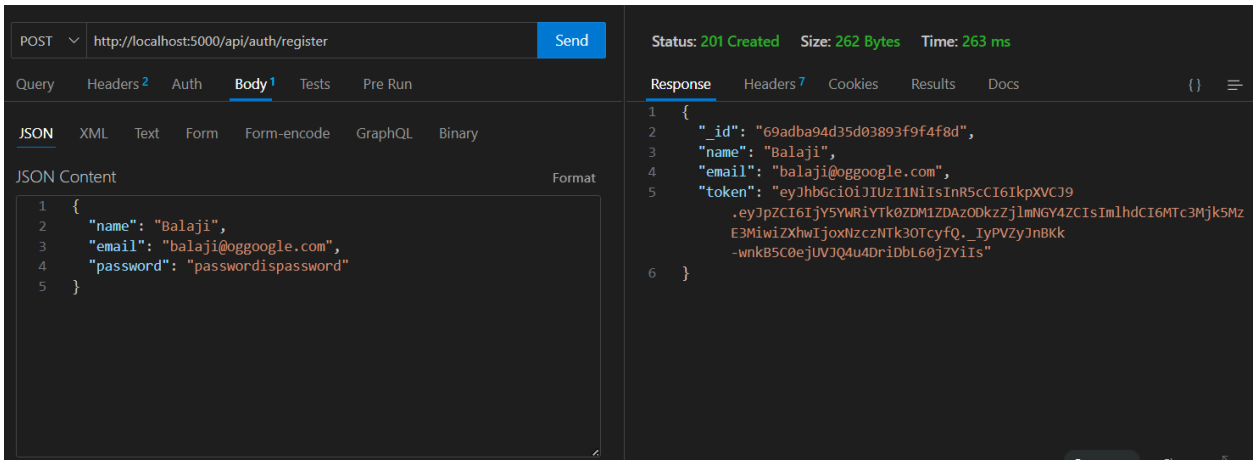
14. API Testing

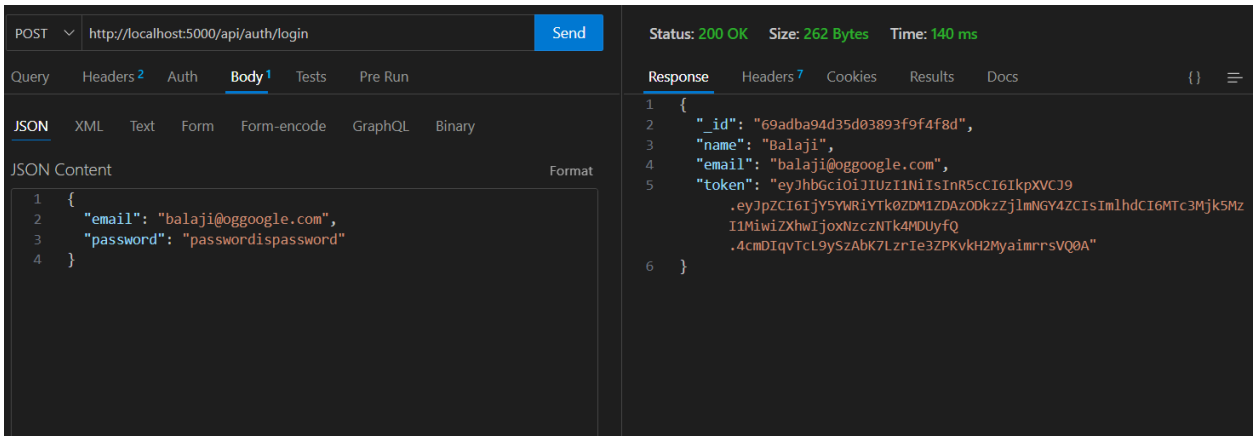
Test APIs using Postman or Thunder Client at http://localhost:5000. This includes registering a user, logging in to generate a JWT token, and using that token to access protected CRUD APIs.

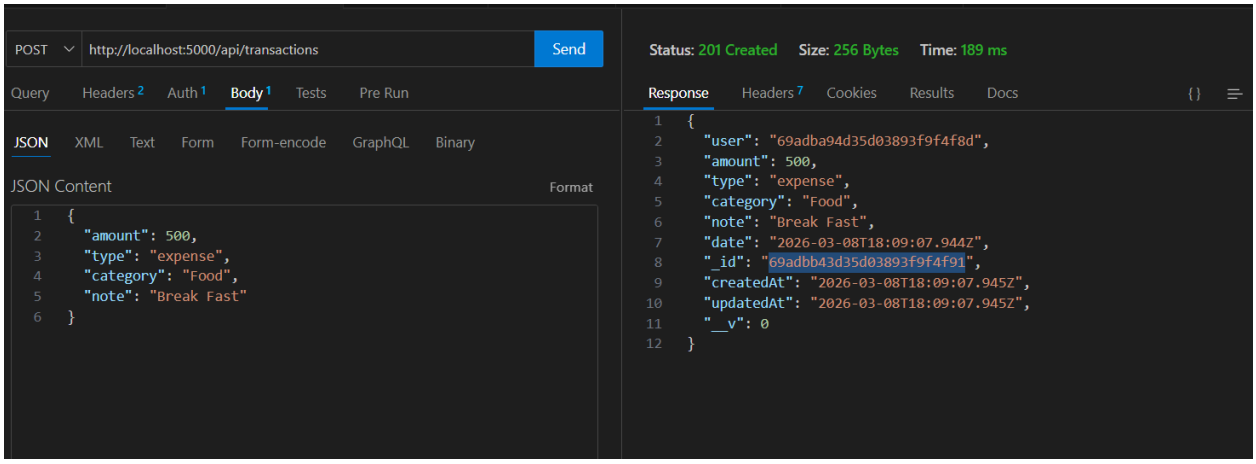
15. Project Execution

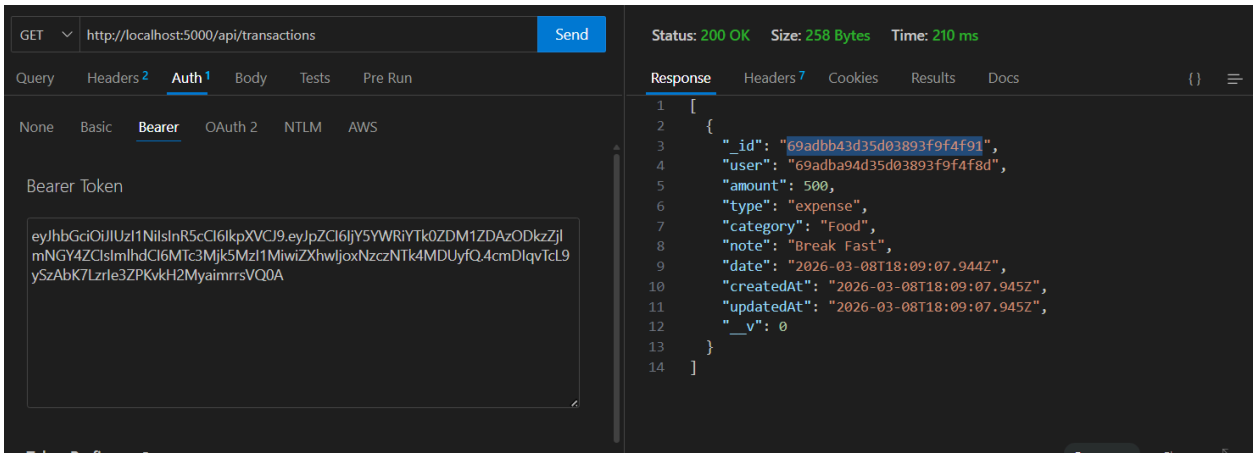
1. Start MongoDB.
2. Run the backend server using npm run dev.

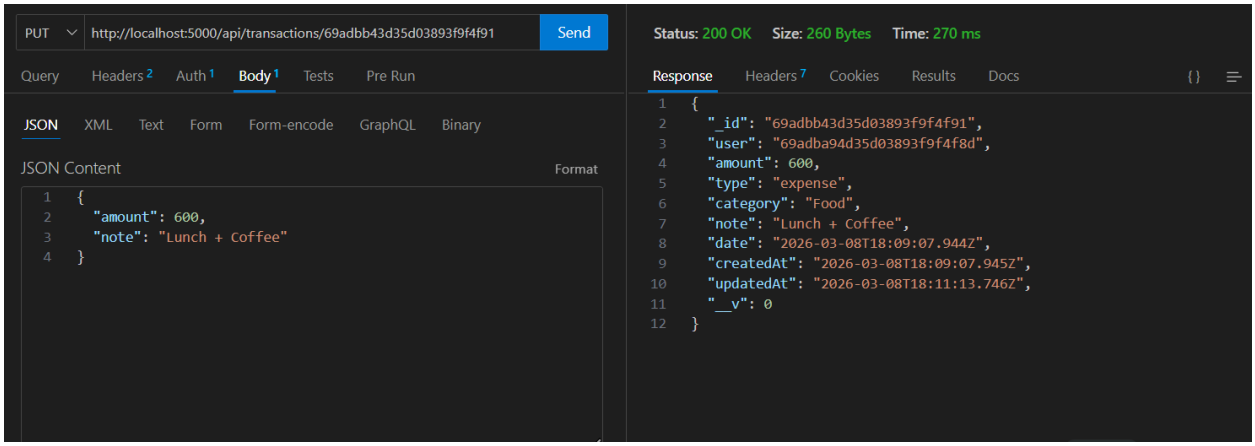
16. Screenshots and Demo

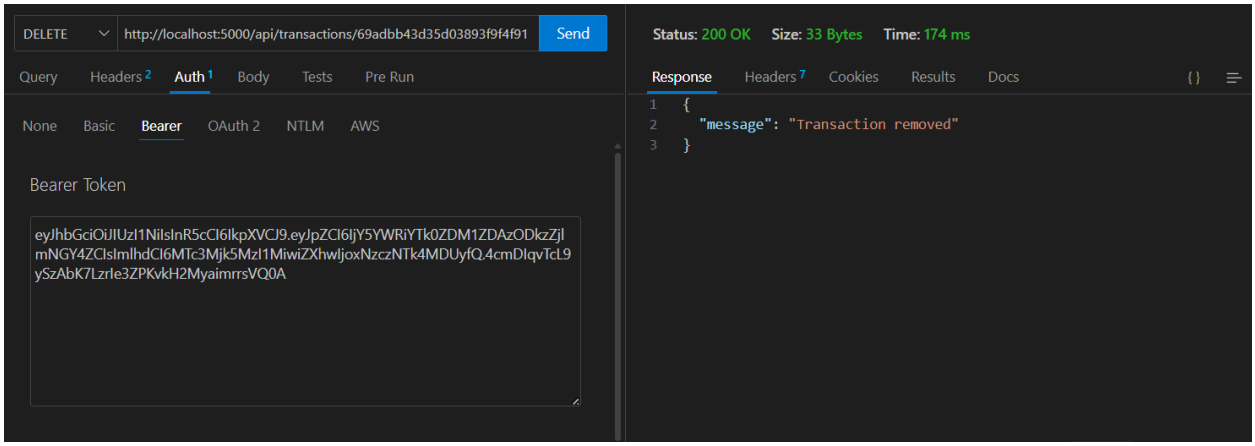












- **Demo Video:** [SpendWise (video).mp4]
- **Github Repo:** [SpendWise]

17. Conclusion

The SpendWise application successfully serves as a robust foundation for learning backend development. By integrating Node.js, Express, and MongoDB, the project demonstrates how to build and structure a secure API from the ground up. Through the implementation of JWT authentication and CRUD operations, it provides practical, real-world experience in handling protected routes and managing relational user data effectively.

Thank You!